

Student Project - UI For (Perhaps More Than) A Single Table

Lindsay Von Tish

lmv9443@nyu.edu

Architecture Overview

This document contains information about the UI and functionality of an application that handles book and member information for a small public library, allowing librarians to manage books and loans while also showing users account information. The planned tables are shown in the below diagram:

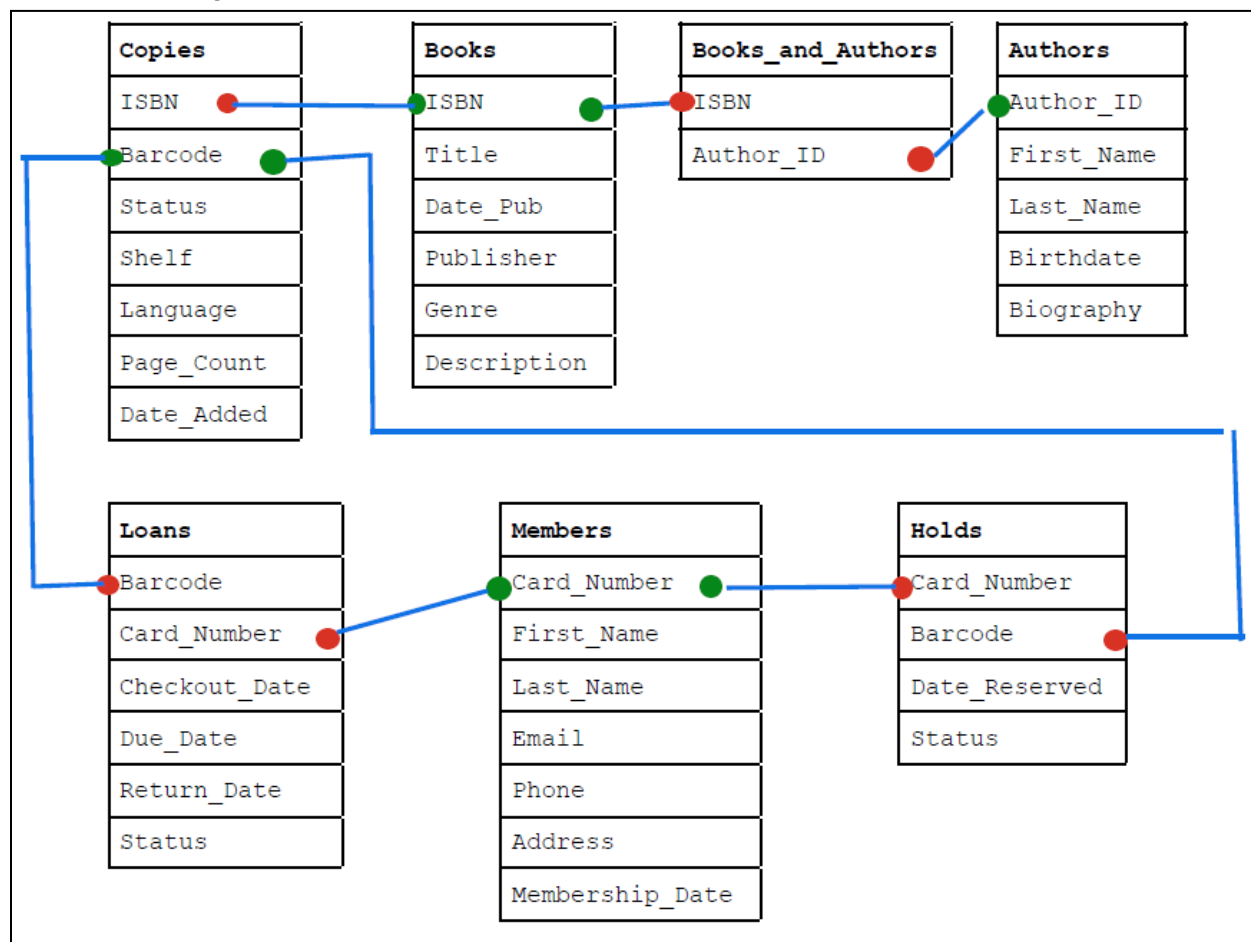


Figure 1 - ER Model

* Please note that 'one' is denoted by a green dot while 'many' is denoted by a red dot.

The 'Books' table contains information about each book in the library's collection while the 'Copies' table holds the data for each individual copy of each book that the library owns. Each 'ISBN' value in the 'Books' table maps to one or more 'ISBN' values in the 'Copies' table, one for each copy. The 'Authors' table holds the information for each individual author and the 'Author_ID' used in the Authors table will match one or more 'Author_ID' values stored in the 'Books_and_Authors' table. The 'ISBN' value in the 'Books_and_Authors' table has a many-to-many relationship with the 'ISBN' value stored in the 'Books' table. This design makes it easy to deal with the fact that many books have multiple authors.

The `Members` table holds the personal information for each of the library's members. Each member is identified by a unique `Card\_Number`. This value has a one-to-many relationship with the corresponding values in the `Loans` and `Holds` tables. The `Loans` table also has a unique `Barcode` value which matches many-to-one with the `Barcode` for each individual book in the `Copies` table, as each copy will have been loaned out multiple times (if my library is successful). Similarly, the `Barcode` value in the `Holds` table also has a many-to-one relationship with the `Barcode` value in the `Copies` table.

Application Development

The application is built in `Python` and uses `Flask` to run its databases. The file tree is shown below:

```
Project/
├── app.py
├── database.db
├── templates/
│   ├── index.html
│   ├── add_author.html
│   ├── add_book.html
│   ├── delete_copy.html
│   └── show_tables.html
```

Figure 2 - Application Files

The Application's `index.html` page takes the user to a basic home page that lists all of the books in the library's collection, as shown below. It also contains links to functionality for librarians to add new books to the collection, add an author to the database, or manually remove copies from the collection.

Library Books	
Add New Book Add Author Delete Copies	
• The Fellowship of the Ring (1) Fantasy God 1123-01-01 Book one in The Lord of the Rings	
• The Two Towers (2) Fantasy God 1951-01-02 The second Lord of the Rings book	
• A Book of Books and Books (1234) Romantasy KDP Baby 2025-01-01 Why are you reading this?	
• The Cult of the Dead Cow (1237) Hackers .ooM 2019-06-04 A biography of the Cult of the Dead Cow.	
View All Tables	

Figure 3 - Index Page

The `View all Tables` button at the bottom takes the user to a quick glance at all of the application databases, which is useful for debugging. This page reveals new information about the database, such as the fact that the library owns two copies of Joseph Menn's "The Cult of the Dead Cow," shown below with the fake ISBN value of 1237.

Database Tables

[Back to Home](#)

Books

ISBN	Title	Date	Publisher	Genre	Description
1	The Fellowship of the Ring	1123-01-01	God	Fantasy	Book one in The Lord of the Rings
2	The Two Towers	1951-01-02	God	Fantasy	The second Lord of the Rings book
1234	A Book of Books and Books	2025-01-01	KDP Baby	Romantasy	Why are you reading this?
1237	The Cult of the Dead Cow	2019-06-04	.ooM	Hackers	A biography of the Cult of the Dead Cow.

Authors

ID	First	Last	Birthdate	Bio
1	Juneau	Lindsay	1967-04-20	Hacker, musician, and semi professional clown.
2	Joseph	Menn	1234-01-02	He writes some cool shit about hackers and the government.
3	Jolkein Rolkein Rolkein	Tolkein	0001-01-01	Tom Bombadil's pen name

Books_and_Authors

ISBN	Author_ID
1	3
2	3
1234	1
1237	2

Copies

Barcode	ISBN	Status	Shelf	Language	Pages	Date Added
1	1	Available	Unknown	English	0	2026-04-06
2	2	Available	Unknown	English	0	2026-04-06
3	1234	Available	Unknown	English	0	2026-04-06
4	1237	Available	Unknown	English	0	2026-04-06
5	1237	Available	Unknown	English	0	2026-04-06

Figure 4 - All Tables

The `Add New Book` page lets librarians add new copies of books to the library collection with a simple form that takes in the book ISBN number, title, publishing date, publisher, genre, and description. It also features a dropdown menu to add authors¹ from the author database, as shown below.

¹ The application functionality currently does not support adding multiple authors to one book. However, the database does, so updating this will be an easy step.

3

The Return of the King

01 / 01 / 1952

God

Fantasy

the Lord of the Rings.

Select Author:

Jolkein Rolkein Rolkein Tolkein (ID: 3) ▾

Add Book

Figure 5 - Dropdown for Adding Books

The `add\_book` functionality, shown in the following figure, allows the librarian to add multiple copies of the same book, even with the same ISBN number. It only adds the copy to the Books table and links the book to the author if the book doesn't already exist in the `Books` table. This allows `Books` to use ISBN as a unique identifier while also generating a unique sequential barcode value used to identify each copy in the `Copies` table.

```
def add_book():
    conn = get_db_connection()

    if request.method == 'POST':
        isbn = request.form['isbn']
        title = request.form['title']
        date_pub = request.form['date_pub']
        publisher = request.form['publisher']
        genre = request.form['genre']
        description = request.form['description']
        author_id = request.form['author_id']

        #conn = get_db_connection()

        # Check that book exists before adding to Books
        existing_book = conn.execute('SELECT * FROM Books WHERE ISBN = ?',
                                     (isbn,)).fetchone()
        if not existing_book:
            # Insert book
            conn.execute('''
                INSERT INTO Books (ISBN, Title, Date_Pub, Publisher, Genre,
                                   Description)
                VALUES (?, ?, ?, ?, ?, ?)
            ''', (isbn, title, date_pub, publisher, genre, description))

            # Link book to author
            conn.execute('''
                INSERT INTO Books_and_Authors (ISBN, Author_ID)
                VALUES (?, ?)
```

```

        '', (isbn, author_id))

# Insert copy (AUTOINCREMENT handles Barcode)
conn.execute('''
    INSERT INTO Copies (ISBN, Status, Shelf, Language, Page_Count,
        Date_Added)
    VALUES (?, ?, ?, ?, ?, DATE('now'))
    '', (isbn, 'Available', 'Unknown', 'English', 0))

conn.commit()
conn.close()

return redirect('/')

# GET request → fetch authors
authors = conn.execute('SELECT * FROM Authors').fetchall()
conn.close()

return render_template('add_book.html', authors=authors)

```

Figure 6 - Adding a Book

After hitting the `Add Book` button, we see the new book, “The Return of the King” added to our library on the index page.

Library Books	
Add New Book Add Author Delete Copies	
The Fellowship of the Ring (1) Fantasy God 1123-01-01 Book one in The Lord of the Rings	
The Two Towers (2) Fantasy God 1951-01-02 The second Lord of the Rings book	
A Book of Books and Books (1234) Romantasy KDP Baby 2025-01-01 Why are you reading this?	
The Cult of the Dead Cow (1237) Hackers .ooM 2019-06-04 A biography of the Cult of the Dead Cow.	
The Return of the King (3) Fantasy God 1952-01-01 The third book in the Lord of the Rings.	
View All Tables	

Figure 7 - New Book Added

Future releases will allow librarians to add new authors while adding new books, but currently the users must visit the `add\_author` page to add new authors to the library database. Each author added to the `Authors` table will appear in the drop down when adding a new book.

```
def add_author():
    if request.method == 'POST':
        first = request.form['first_name']
        last = request.form['last_name']
        birthdate = request.form['birthdate']
        bio = request.form['biography']

        conn = get_db_connection()

        cursor = conn.execute('''
            INSERT INTO Authors (First_Name, Last_Name, Birthdate, Biography)
            VALUES (?, ?, ?, ?)
            ''', (first, last, birthdate, bio))

        author_id = cursor.lastrowid
        print("Inserted ID:", cursor.lastrowid)

        conn.commit()
        conn.close()

        return redirect('/')

    return render_template('add_author.html')
```

Figure 8 - Adding an Author

Each author has a unique ID similar to the individual copies Barcode values.

The delete copy page shows all of the book and copy information available for each copy.

Barcode	ISBN	Title	Author	Status	Shelf	Language	Pages	Date Added	Action
1	1	The Fellowship of the Ring	Jolkein Rolkein Rolkein Tolkein	Available	Unknown	English	0	2026-04-06	Delete
2	2	The Two Towers	Jolkein Rolkein Rolkein Tolkein	Available	Unknown	English	0	2026-04-06	Delete
3	1234	A Book of Books and Books	Juneau Lindsay	Available	Unknown	English	0	2026-04-06	Delete
4	1237	The Cult of the Dead Cow	Joseph Menn	Available	Unknown	English	0	2026-04-06	Delete
5	1237	The Cult of the Dead Cow	Joseph Menn	Available	Unknown	English	0	2026-04-06	Delete
6	3	The Return of the King	Jolkein Rolkein Rolkein Tolkein	Available	Unknown	English	0	2026-04-06	Delete

Figure 9 - Delete Copy Page

This is made possible by a join between the `Books` and `Copies` tables, as shown in the following code snippet:

```
@app.route('/delete_copy', methods=('GET', 'POST'))
def delete_copy():
    conn = get_db_connection()

    if request.method == 'POST':
        barcode = request.form['barcode']
        # Check if copy exists
        copy = conn.execute('SELECT * FROM Copies WHERE Barcode = ?',
(barcode,)).fetchone()
        if copy:
            conn.execute('DELETE FROM Copies WHERE Barcode = ?', (barcode,))
            conn.commit()
            message = f"Copy with Barcode {barcode} deleted."
        else:
            message = f"No copy found with Barcode {barcode}."
    else:
        message = None

    # Fetch all copies with book title and author
    copies = conn.execute('''
        SELECT
            c.Barcode,
            c.ISBN,
            c.Status,
            c.Shelf,
            c.Language,
            c.Page_Count,
            c.Date_Added,
            b.Title,
            a.First_Name || ' ' || a.Last_Name AS Author
        FROM Copies c
        JOIN Books b ON c.ISBN = b.ISBN
        LEFT JOIN Books_and_Authors ba ON b.ISBN = ba.ISBN
        LEFT JOIN Authors a ON ba.Author_ID = a.Author_ID
        ORDER BY c.Barcode
    ''').fetchall()

    conn.close()
    return render_template('delete_copy.html', copies=copies, message=message)
```

Figure 8 - Adding an Author

If we delete, for example, a stunning piece of literature such as “A Book of Books and Books,” it will be removed from the copy tables, but the author information will remain.

Appendices

Note, for some reason peerceptive isn't letting me upload multiple files. To deal with that, I have most of the interesting code below in plaintext. If you would like to download and execute the code yourself, please [refer to the GitHub](#). An additional copy of this document, along with a file named "Project.7z" that contains all of the source code will be made available in [this Google Drive Folder](#).

Appendix A: app.py

```
from flask import Flask, render_template, request, redirect
import sqlite3

app = Flask(__name__)

def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db_connection()
    # Create Books
    conn.execute('''
        CREATE TABLE IF NOT EXISTS Books (
            ISBN TEXT PRIMARY KEY,
            Title TEXT NOT NULL,
            Date_Pub TEXT,
            Publisher TEXT,
            Genre TEXT,
            Description TEXT
        )
    ''')

    # Create Authors
    conn.execute('''
        CREATE TABLE IF NOT EXISTS Authors (
            Author_ID INTEGER PRIMARY KEY AUTOINCREMENT,
            First_Name TEXT,
            Last_Name TEXT,
            Birthdate TEXT,
            Biography TEXT
        )
    ''')

    # Junction table Books_and_Authors
    conn.execute('''
        CREATE TABLE IF NOT EXISTS Books_and_Authors (
            ISBN TEXT,
            Author_ID INTEGER,
            FOREIGN KEY (ISBN) REFERENCES Books (ISBN),
            FOREIGN KEY (Author_ID) REFERENCES Authors (Author_ID)
        )
    ''')
```



```

'''
# Copies Table
conn.execute('''
    CREATE TABLE IF NOT EXISTS Copies (
        Barcode INTEGER PRIMARY KEY AUTOINCREMENT,
        ISBN TEXT,
        Status TEXT,
        Shelf TEXT,
        Language TEXT,
        Page_Count INTEGER,
        Date_Added TEXT,
        FOREIGN KEY (ISBN) REFERENCES Books (ISBN)
    )
''')

conn.commit()
conn.close()

@app.route('/')
def index():
    conn = get_db_connection()
    books = conn.execute('SELECT * FROM Books').fetchall()
    conn.close()
    return render_template('index.html', books=books)

@app.route('/add', methods=('GET', 'POST'))
def add_book():
    conn = get_db_connection()

    if request.method == 'POST':
        isbn = request.form['isbn']
        title = request.form['title']
        date_pub = request.form['date_pub']
        publisher = request.form['publisher']
        genre = request.form['genre']
        description = request.form['description']
        author_id = request.form['author_id']

        #conn = get_db_connection()

        # Check that book exists before adding to Books
        existing_book = conn.execute('SELECT * FROM Books WHERE ISBN = ?',
(isbn,)).fetchone()
        if not existing_book:
            # Insert book
            conn.execute('''
                INSERT INTO Books (ISBN, Title, Date_Pub, Publisher, Genre,
Description)
                VALUES (?, ?, ?, ?, ?, ?)
            ''', (isbn, title, date_pub, publisher, genre, description))

            # Link book to author
            conn.execute('''
                INSERT INTO Books_and_Authors (ISBN, Author_ID)
                VALUES (?, ?)
            ''')

```

```

        '', (isbn, author_id))

    # Generate Barcode
    #count = conn.execute('SELECT COUNT(*) FROM Copies').fetchone()[0]
    #barcode = count + 1

    # Insert copy (AUTOINCREMENT handles Barcode)
    conn.execute('''
        INSERT INTO Copies (ISBN, Status, Shelf, Language, Page_Count,
Date_Added)
        VALUES (?, ?, ?, ?, ?, DATE('now'))
        '', (isbn, 'Available', 'Unknown', 'English', 0))

    conn.commit()
    conn.close()

    return redirect('/')

# GET request → fetch authors
authors = conn.execute('SELECT * FROM Authors').fetchall()
conn.close()

return render_template('add_book.html', authors=authors)

@app.route('/add_author', methods=('GET', 'POST'))
def add_author():
    if request.method == 'POST':
        first = request.form['first_name']
        last = request.form['last_name']
        birthdate = request.form['birthdate']
        bio = request.form['biography']

        conn = get_db_connection()

        # Generate Author_ID
        #count = conn.execute('SELECT COUNT(*) FROM Authors').fetchone()[0]
        #author_id = count + 1

        cursor = conn.execute('''
            INSERT INTO Authors (First_Name, Last_Name, Birthdate, Biography)
            VALUES (?, ?, ?, ?)
            '', (first, last, birthdate, bio))

        author_id = cursor.lastrowid
        print("Inserted ID:", cursor.lastrowid)

        conn.commit()
        conn.close()

        return redirect('/')

    return render_template('add_author.html')

@app.route('/tables')
def show_tables():

```

```

conn = get_db_connection()

books = conn.execute('SELECT * FROM Books').fetchall()
authors = conn.execute('SELECT * FROM Authors').fetchall()
books_authors = conn.execute('SELECT * FROM Books_and_Authors').fetchall()
copies = conn.execute('SELECT * FROM Copies').fetchall()

conn.close()

return render_template(
    'show_tables.html',
    books=books,
    authors=authors,
    books_authors=books_authors,
    copies=copies
)

'''
@app.route('/delete_copy', methods=('GET', 'POST'))
def delete_copy():
    conn = get_db_connection()

    if request.method == 'POST':
        barcode = request.form['barcode']
        # Check if copy exists
        copy = conn.execute('SELECT * FROM Copies WHERE Barcode = ?',
(barcode,)).fetchone()
        if copy:
            conn.execute('DELETE FROM Copies WHERE Barcode = ?', (barcode,))
            conn.commit()
            message = f"Copy with Barcode {barcode} deleted."
        else:
            message = f"No copy found with Barcode {barcode}."
    else:
        message = None

    # Fetch all copies to display in the table
    copies = conn.execute('SELECT * FROM Copies').fetchall()
    conn.close()

    return render_template('delete_copy.html', copies=copies, message=message)
'''

@app.route('/delete_copy', methods=('GET', 'POST'))
def delete_copy():
    conn = get_db_connection()

    if request.method == 'POST':
        barcode = request.form['barcode']
        # Check if copy exists
        copy = conn.execute('SELECT * FROM Copies WHERE Barcode = ?',
(barcode,)).fetchone()
        if copy:
            conn.execute('DELETE FROM Copies WHERE Barcode = ?', (barcode,))
            conn.commit()
            message = f"Copy with Barcode {barcode} deleted."

```

```

        else:
            message = f"No copy found with Barcode {barcode}."
    else:
        message = None

    # Fetch all copies with book title and author
    copies = conn.execute('''
        SELECT
            c.Barcode,
            c.ISBN,
            c.Status,
            c.Shelf,
            c.Language,
            c.Page_Count,
            c.Date_Added,
            b.Title,
            a.First_Name || ' ' || a.Last_Name AS Author
        FROM Copies c
        JOIN Books b ON c.ISBN = b.ISBN
        LEFT JOIN Books_and_Authors ba ON b.ISBN = ba.ISBN
        LEFT JOIN Authors a ON ba.Author_ID = a.Author_ID
        ORDER BY c.Barcode
    ''').fetchall()

    conn.close()
    return render_template('delete_copy.html', copies=copies, message=message)

if __name__ == '__main__':
    init_db()
    app.run(debug=True)

```

Appendix B: index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Library</title>
</head>
<body>
    <h1>Library Books</h1>

    <a href="/add">Add New Book</a>
    <a href="/add_author">Add Author</a>
    <a href="/delete_copy">Delete Copies</a>

    <ul>
        {% for book in books %}
            <li>
                <strong>{{ book['Title'] }}</strong> ({{ book['ISBN'] }})<br>
                {{ book['Genre'] }} | {{ book['Publisher'] }}<br>
                {{ book['Date_Pub'] }}<br>
                {{ book['Description'] }}
            </li>
        {% endfor %}
    </ul>

```

```

        <hr>
        {% endfor %}
    </ul>

    <a href="/tables">View All Tables</a>

</body>
</html>

```

Appendix C: add_author.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Add Author</title>
</head>
<body>
    <h1>Add Author</h1>

    <form method="post">
        <input type="text" name="first_name" placeholder="First Name"><br>
        <input type="text" name="last_name" placeholder="Last Name"><br>
        <input type="date" name="birthdate"><br>

        <textarea name="biography" placeholder="Biography"></textarea><br>

        <button type="submit">Add Author</button>
    </form>

    <a href="/">Back</a>
</body>
</html>

```

Appendix D: add_book.html

```

<form method="post">
    <input type="text" name="isbn" placeholder="ISBN" required><br>
    <input type="text" name="title" placeholder="Title" required><br>
    <input type="date" name="date_pub"><br>
    <input type="text" name="publisher" placeholder="Publisher"><br>
    <input type="text" name="genre" placeholder="Genre"><br>

    <textarea name="description" placeholder="Description"></textarea><br>

    <label>Select Author:</label><br>
    <select name="author_id" required>
        <option value="">-- Select Author --</option>
        {% for author in authors %}
            <option value="{{ author['Author_ID'] }}">
                {{ author['First_Name'] }} {{ author['Last_Name'] }} (ID: {{
author['Author_ID'] }})
            </option>
        {% endfor %}
    </select>
</form>

```

```

        </select><br><br>

        <button type="submit">Add Book</button>
    </form>

```

Appendix D: delete_copy.html

```

<table>
  <tr>
    <th>Barcode</th>
    <th>ISBN</th>
    <th>Title</th>
    <th>Author</th>
    <th>Status</th>
    <th>Shelf</th>
    <th>Language</th>
    <th>Pages</th>
    <th>Date Added</th>
    <th>Action</th>
  </tr>
  {% for c in copies %}
  <tr>
    <td>{{ c['Barcode'] }}</td>
    <td>{{ c['ISBN'] }}</td>
    <td>{{ c['Title'] }}</td>
    <td>{{ c['Author'] or 'Unknown' }}</td>
    <td>{{ c['Status'] }}</td>
    <td>{{ c['Shelf'] }}</td>
    <td>{{ c['Language'] }}</td>
    <td>{{ c['Page_Count'] }}</td>
    <td>{{ c['Date_Added'] }}</td>
    <td>
      <form method="post" style="margin:0;">
        <input type="hidden" name="barcode" value="{{ c['Barcode']
    }}">
        <button type="submit">Delete</button>
      </form>
    </td>
  </tr>
  {% endfor %}
</table>

```

Appendix E: show_tables.html

```

<!DOCTYPE html>
<html>
<head>
  <title>All Tables</title>
  <style>
    table {
      border-collapse: collapse;
      margin-bottom: 30px;
    }

```

```

        th, td {
            border: 1px solid black;
            padding: 6px 10px;
        }
    </style>
</head>
<body>

<h1>Database Tables</h1>

<a href="/">Back to Home</a>

<h2>Books</h2>
<table>
    <tr>

<th>ISBN</th><th>Title</th><th>Date</th><th>Publisher</th><th>Genre</th><th>De
scription</th>
    </tr>
    {% for b in books %}
    <tr>
        <td>{{ b['ISBN'] }}</td>
        <td>{{ b['Title'] }}</td>
        <td>{{ b['Date_Pub'] }}</td>
        <td>{{ b['Publisher'] }}</td>
        <td>{{ b['Genre'] }}</td>
        <td>{{ b['Description'] }}</td>
    </tr>
    {% endfor %}
</table>

<h2>Authors</h2>
<table>
    <tr>
        <th>ID</th><th>First</th><th>Last</th><th>Birthdate</th><th>Bio</th>
    </tr>
    {% for a in authors %}
    <tr>
        <td>{{ a['Author_ID'] }}</td>
        <td>{{ a['First_Name'] }}</td>
        <td>{{ a['Last_Name'] }}</td>
        <td>{{ a['Birthdate'] }}</td>
        <td>{{ a['Biography'] }}</td>
    </tr>
    {% endfor %}
</table>

<h2>Books_and_Authors</h2>
<table>
    <tr>
        <th>ISBN</th><th>Author_ID</th>
    </tr>
    {% for ba in books_authors %}
    <tr>
        <td>{{ ba['ISBN'] }}</td>
        <td>{{ ba['Author_ID'] }}</td>
    </tr>
    {% endfor %}
</table>

```

```
</tr>
  {% endfor %}
</table>

<h2>Copies</h2>
<table>
  <tr>
    <th>Barcode</th><th>ISBN</th><th>Status</th><th>Shelf</th>
    <th>Language</th><th>Pages</th><th>Date Added</th>
  </tr>
  {% for c in copies %}
  <tr>
    <td>{{ c['Barcode'] }}</td>
    <td>{{ c['ISBN'] }}</td>
    <td>{{ c['Status'] }}</td>
    <td>{{ c['Shelf'] }}</td>
    <td>{{ c['Language'] }}</td>
    <td>{{ c['Page_Count'] }}</td>
    <td>{{ c['Date_Added'] }}</td>
  </tr>
  {% endfor %}
</table>

</body>
</html>
```